1024.3 AI-Enabled IT Support Capstone

Migration & Stabilization

Setting Up & Migrating a Windows Server and Client

AMANDA KONDRAT'YEV

Instructor: Luke Watson

Per Scholas | Houston

Date: June 25, 2026

Table of Contents

Project Overview

This report documents the setup, migration, and stabilization of a Windows Server and Windows 11 client environment, written so a new junior IT support specialist could follow the process. It covers building and validating a stable baseline, executing a required migration, investigating the issues that followed, restoring stability, and reporting the outcome.

The scenario simulates a small enterprise: a Windows Server 2022 domain controller (MIG-SRV01) hosting Active Directory, DNS, and a shared file service for the domain migration.local (NetBIOS MIGRATION), with a Windows 11 client (MIG-CLI01) joined to that domain. Both machines run in VirtualBox on an isolated internal network, so the environment is fully self-contained. The work proceeds through five phases: building and validating the baseline (Phase 1), executing the required migration event (Phase 2), investigating the issues that follow (Phase 3), applying verified fixes to restore stability (Phase 4), and reporting the outcome with reflection (Phase 5).

Project repository: https://github.com/manderwall/it-support-capstone

Phase 1 — Environment & Baseline

This phase builds and validates a stable enterprise environment before any migration takes place. Each step below is documented with the supporting screenshots collected in Appendix A.

Downloading & Installing VirtualBox

Oracle VirtualBox was downloaded from the official site (virtualbox.org) along with the matching VirtualBox Extension Pack, version 7.2.10, and both were installed on the host laptop. Hardware virtualization (VT-x / AMD-V) was confirmed enabled. The Extension Pack was verified active in the Extension Pack Manager and the installed build confirmed under Help > About (Appendix A, Figures 1–2).

Downloading Files

The installation media and scripts were gathered next: the Windows Server 2022 ISO, the Windows 11 ISO, and the instructor-provided Phase 1 verification and Phase 2 migration scripts, all downloaded from the course links and saved locally before any VM was created.

Setting Up Windows Server

A virtual machine named MIG-SRV01 was created to the required specification — Windows Server 2022 (64-bit), 2 CPUs, 6 GB RAM, an 80 GB virtual disk, with Network Adapter 1 attached to an internal-only network so the lab stays isolated (Appendix A, Figure 3). Windows Server 2022 (Desktop Experience) was installed from the ISO and a local Administrator password set; the server reached the desktop and Server Manager on first sign-in (Figure 4). VirtualBox Guest Additions were installed and bidirectional clipboard enabled so the configuration scripts could be transferred into the VM.

Setting Up Windows 11 (Client)

A second virtual machine, MIG-CLI01, was created for the client — Windows 11 (64-bit), 2 CPUs, 4 GB RAM, a 60 GB disk — on the same internal network, with EFI, a TPM 2.0 chip, and Secure Boot enabled to satisfy Windows 11's requirements (Appendix A, Figure 5). Windows 11 was installed and a local account created. A clean snapshot of each VM was captured before any configuration, so the environment could be restored to a known-good baseline if needed (Figure 6).

File Services Baseline Configuration

Required values — confirm each matches your build:

Item	Required value	Done
Domain (FQDN)	migration.local	☑
NetBIOS	MIGRATION	☑
Domain Admin / User	MigrationAdmin / MigrationUser	☑
Groups	IT-Admins, Domain-Users, FileShareUsers	☑
Shared folder	C:\SharedData	☑
SMB share	SharedData (FileShareUsers has access)	☑

On the domain controller a folder, C:\SharedData, was created and shared over SMB as SharedData, and the security group FileShareUsers was granted access (Change at the share level and Modify on NTFS). The resulting accounts, groups, share, and permissions are shown in Appendix A, Figure 9.

Script Execution

AI-assisted. The full PowerShell setup script is included in Appendix B.

An AI-assisted PowerShell script was used to automate the configuration; it was reviewed line by line before use and run from an elevated PowerShell session, server first and then client. On MIG-SRV01 the script set a static IP (192.168.50.10) and DNS, renamed the computer, installed the Active Directory Domain Services and DNS roles, and promoted the server to a domain controller for the new forest migration.local (NetBIOS MIGRATION). It then created the accounts MigrationAdmin (added to Domain Admins) and MigrationUser, the groups IT-Admins, Domain-Users, and FileShareUsers, and the SharedData share. On MIG-CLI01 the script set a static IP (192.168.50.20), pointed DNS at the domain controller (192.168.50.10), renamed the computer, and joined it to migration.local. The evidence captured during execution covers the server IP/DNS (Figure 7), the AD DS role install (Figure 8), the accounts/groups/share (Figure 9), and the client IP/DNS (Figure 10). The client was then signed into as MIGRATION\MigrationUser, confirming centralized authentication (Figure 11), and that account opened and wrote to \\MIG-SRV01\SharedData (Figure 12). Finally, the instructor's Phase 1 verification script returned PASS on all checks (Figure 13).

Issues & Resolutions

Two issues arose during Phase 1, each documented below.

Issue 1 — Windows 11 client would not boot.

What was the issue?	The Windows 11 client VM showed only a black screen at startup and never reached Setup, even though the Windows Server VM booted normally.
What was your approach?	I confirmed the VM's firmware was correct (EFI, TPM 2.0, and Secure Boot all enabled as Windows 11 requires), then looked at the host, where Windows virtualization features (Virtual Machine Platform / Hyper-V) were competing with VirtualBox for hardware virtualization.
What other solutions did you consider or try?	Toggling Secure Boot on the VM made no difference — the conflict was on the host side, not in the VM's settings.
What was the final resolution / outcome?	I disabled the conflicting host virtualization feature and restarted the host; the Windows 11 VM then booted to Setup and installed normally.

Issue 2 — DNS verification failure.

What was the issue?	On its first run the Phase 1 verification script reported a single failure — "DNS is not pointing to this server" — while every other check passed.
What was your approach?	I inspected the domain controller's DNS client settings and found its preferred DNS server was set to the loopback address (127.0.0.1) instead of the server's own static IP.
What other solutions did you consider or try?	Loopback is a functional DNS setting for a domain controller and name resolution already worked, so rather than rebuild or make any broader change, I applied the minimal targeted fix the check required: re-pointing DNS to the static IP.

What was the final resolution / outcome?	I set the server’s preferred DNS to its static IP (192.168.50.10) with Set-DnsClientServerAddress, then re-ran the verification script; it returned PASS on every check (Appendix A, Figure 13).

Phase 2 — Migration Event

With both virtual machines snapshotted as a rollback point, the centrally provided migration script was executed once on the domain controller (MIG-SRV01) as Administrator. The script ran to completion (Appendix A, Figure 14). In keeping with the phase's intent, no troubleshooting or configuration changes were made during this step — the migration was run as issued and its completion recorded, so that any resulting instability could be investigated cleanly in Phase 3.

Phase 3 — Incident Response

After the migration, six incidents were reported. Each was investigated on the affected machine using read-only diagnostics — Event Viewer, gpresult, service and permission checks, DNS lookups, and secure-channel and time tests — with the output captured to transcripts and screenshots (Appendix A, Figures 15–17). Consistent with the phase's scope, this was diagnosis only; no fixes were applied. The table below records the reported symptom and what the evidence showed for each ticket.

Ticket	Reported symptom	What the evidence showed (diagnosis only)
INC-001	Access Denied to shared folder	From the client, \\MIG-SRV01\SharedData returns “Access is denied.” On the server the share-level permission still grants FileShareUsers Change, but the NTFS permissions on C:\SharedData no longer include the FileShareUsers “Modify” entry set in Phase 1, so access is blocked at the file-system layer.
INC-002	Slow login after migration	The Group Policy operational log shows a new domain policy, Baseline-User-Policy, applying during user logon. The added user policy is the change introduced at sign-in.
INC-003	Desktop settings not applying	Tied to the same Baseline-User-Policy GPO introduced by the migration; the client's Group Policy processing and applicable-GPO list were captured for comparison.
INC-004	Printer no longer available	On the client only “Microsoft Print to PDF” remains installed while the Print Spooler service is running — so the previously available printer was removed, not disabled by a stopped service.
INC-005	System feels sluggish	Several Automatic services were found stopped (Delivery Optimization, edgeupdate, MapsBroker), and Microsoft Defender (MsMpEng) was the top memory consumer at the time of capture.
INC-006	Intermittent file access	DNS (192.168.50.10), the domain secure channel, and time synchronization all tested healthy on the client, so name resolution and trust are not the cause; this incident traces back to the same share / NTFS permission issue as INC-001.

A clear pattern emerged: the migration altered access control and Group Policy rather than breaking core infrastructure. The domain, DNS, secure channel, and time services all remained healthy, which steered the investigation toward file-system permissions (INC-001 / INC-006) and the newly introduced user policy (INC-002 / INC-003) rather than a network or trust failure.

Project Status & Scope

This report covers the work completed through Phase 3. The Phase 1 baseline was built and independently verified, the Phase 2 migration was executed on the server, and the Phase 3 incident response was completed — all six reported incidents were investigated on both machines with evidence captured. The submission stops before Phase 4 (applying the corrective fixes) and Phase 5 (the executive summary and final reflection).

Why This Submission Concludes After Phase 3

The environment is fully preserved: clean and pre-migration snapshots exist for both machines, and all scripts, transcripts, and screenshots are saved — so Phases 4 and 5 can be resumed at any time from a known-good state. Concluding here is a matter of available time, not capability. Over the project the centrally provided migration script was reissued more than once, and a further revision arrived after the migration and the full investigation were already complete; adopting it would have meant re-running the migration and repeating the entire investigation. Given the project timeline and the repeated rework those changes required, I chose to submit the completed, evidence-backed work through Phase 3. Because the Phase 3 diagnoses already identify what each corrective fix would target, the remaining Phase 4 work is well-scoped and can be carried out from the saved rollback point.

What Was Accomplished

- Built and verified the full Phase 1 baseline — two VMs, Active Directory, DNS, accounts and groups, and a permissioned SMB share — using reviewed, AI-assisted automation; every verification check passed.
- Executed the Phase 2 migration on the domain controller and recorded its completion.
- Investigated all six post-migration incidents (INC-001 through INC-006) on both machines, captured transcripts and screenshots, and identified the evidence-based cause of each.
- Diagnosed and corrected a DNS misconfiguration during the Phase 1 build, then re-verified to a full pass.
- Produced reusable, documented automation for the build and for the Phase 3 diagnostics (Appendix C), with automatic evidence capture throughout.

Reflection

Two moments shaped how I worked. In Phase 1, the verification script reported that DNS was not pointing to the server even though name resolution seemed fine; rather than rebuild or guess, I inspected the DNS client settings, found the loopback address in place of the static IP, made that one change, and re-verified. In Phase 3, the same discipline applied at a larger scale: instead of assuming the migration had broken the network, I let the evidence narrow the problem — the domain, DNS, and secure channel were healthy, which pointed cleanly at permissions and a new Group Policy object. Let the evidence define the problem, make the smallest correct change, then confirm it. That habit also governed how I used AI throughout: as a fast drafting assistant whose output I treated as a starting point to verify, never as an authority to trust unread.

Appendix A — Screenshots

All Phase 1 evidence, captured during the build and verification, grouped by stage.

Setup & Installation

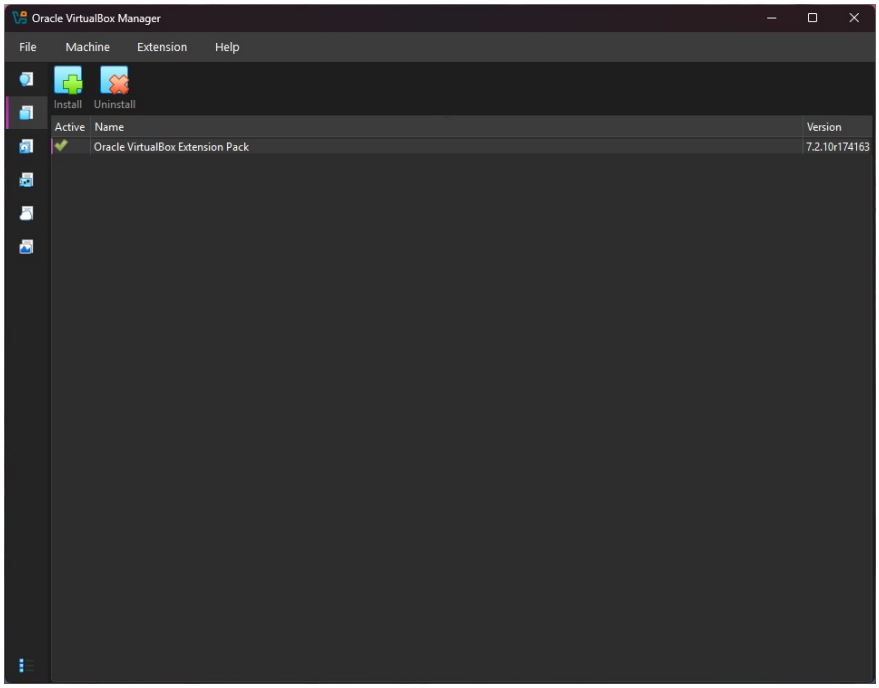

Figure 1: VirtualBox Extension Pack 7.2.10 installed and active

Figure 2: VirtualBox version confirmed (Help > About)

Virtual Machines & Snapshot

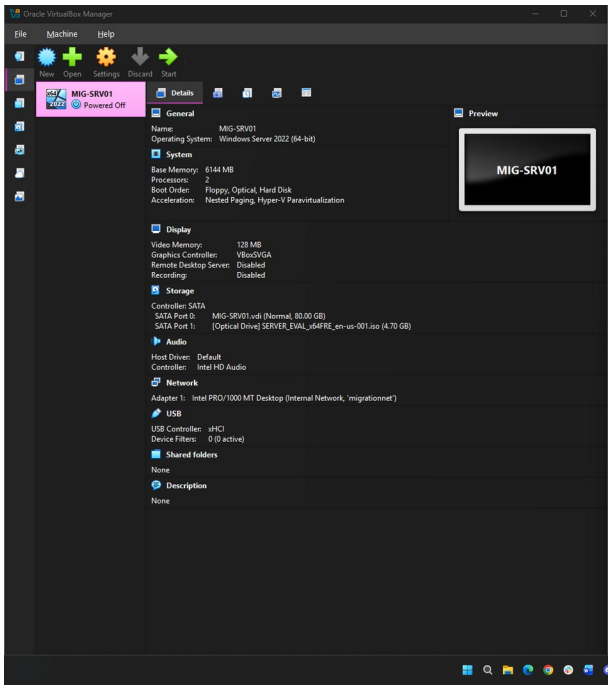

Figure 3: MIG-SRV01 settings: 2 CPU, 6 GB RAM, 80 GB disk, Internal Network

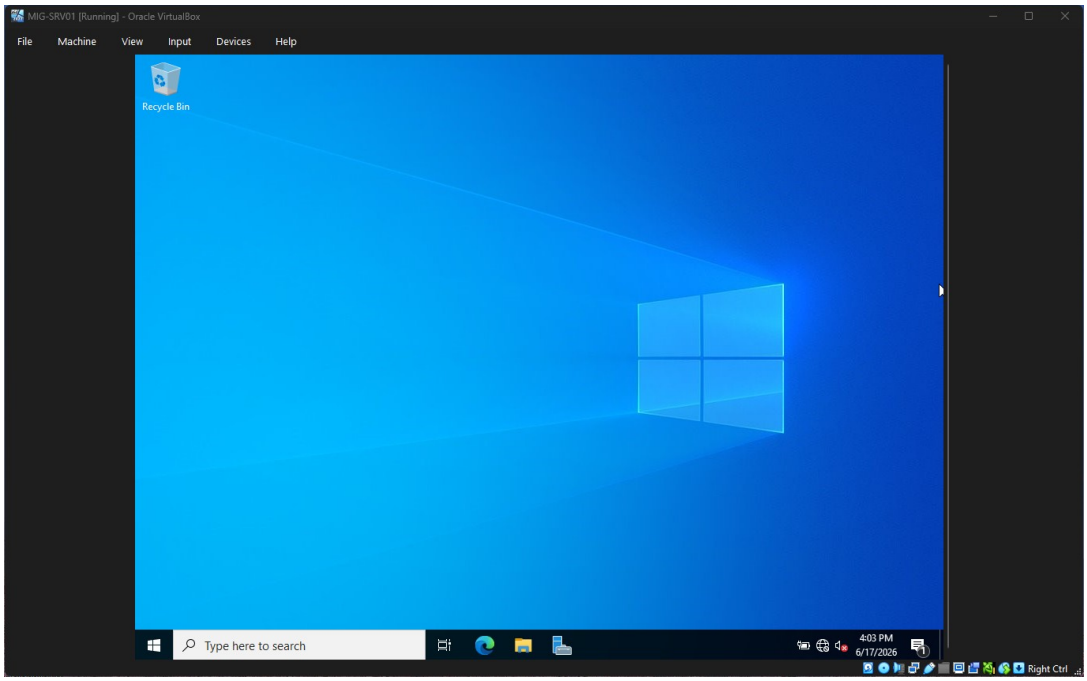

Figure 4: Windows Server 2022 desktop and Server Manager on first sign-in

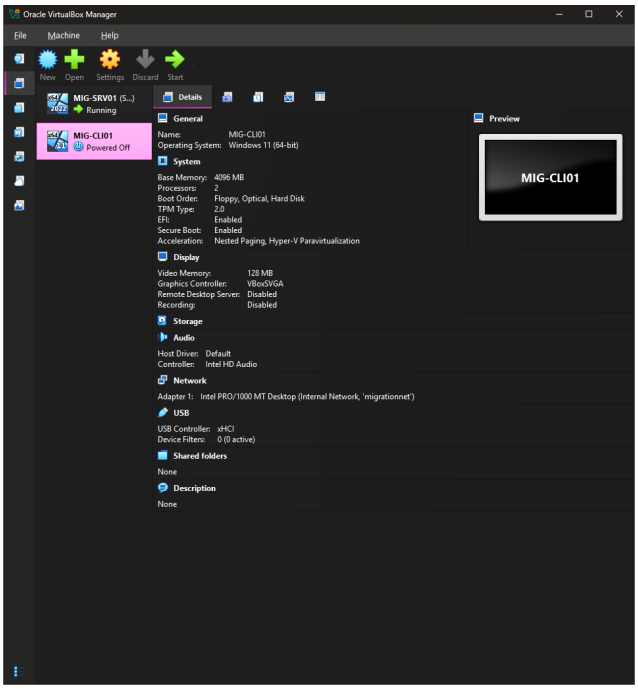

Figure 5: MIG-CLI01 settings: EFI, TPM 2.0, and Secure Boot enabled

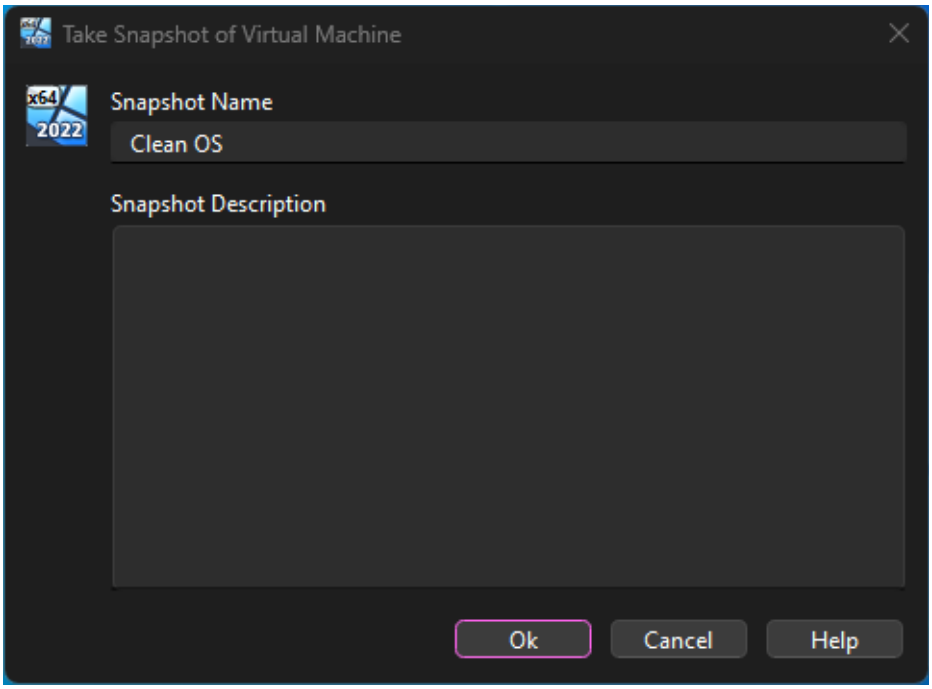

Figure 6: Clean OS snapshot captured before configuration

Server Configuration

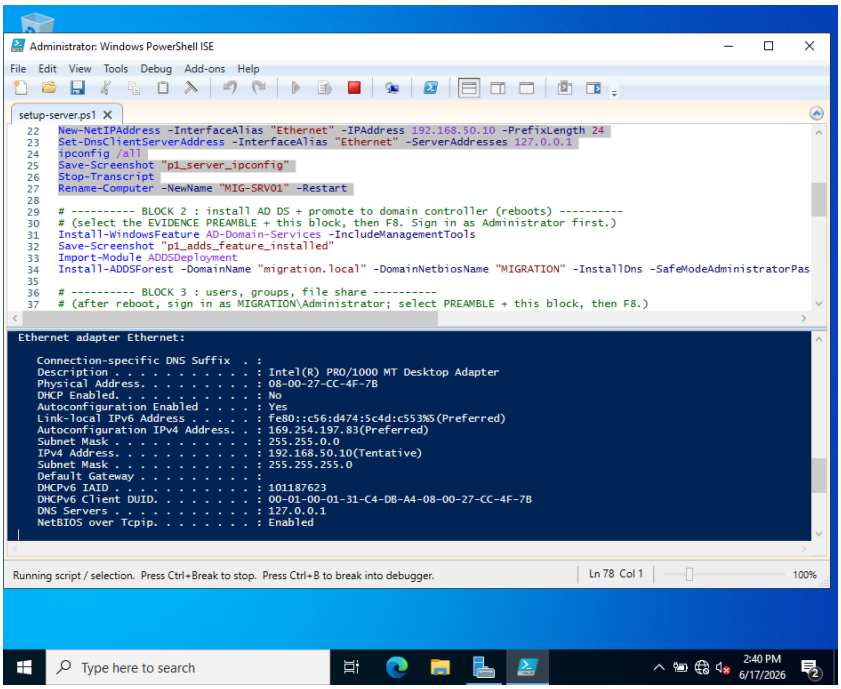

Figure 7: Server static IP 192.168.50.10 and DNS (ipconfig /all)

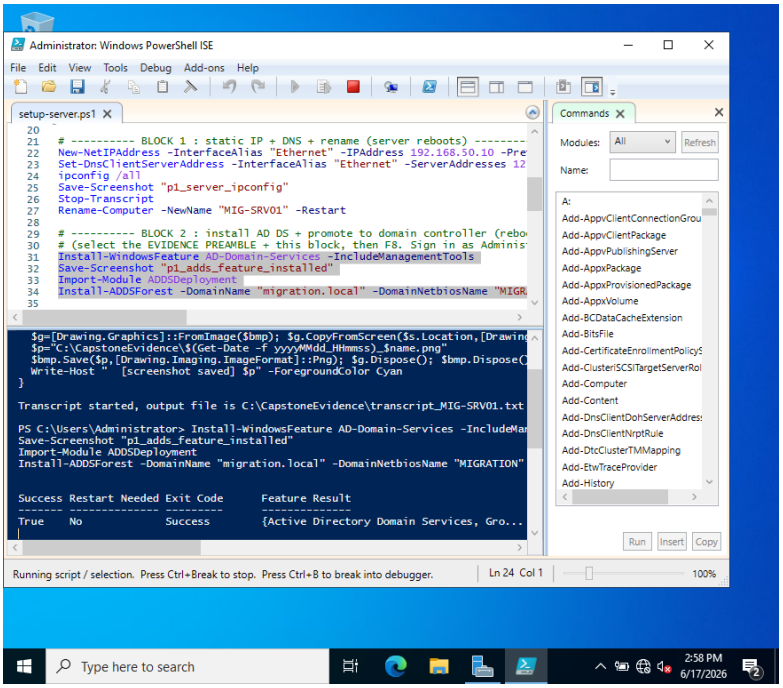

Figure 8: Active Directory Domain Services role installed on the server

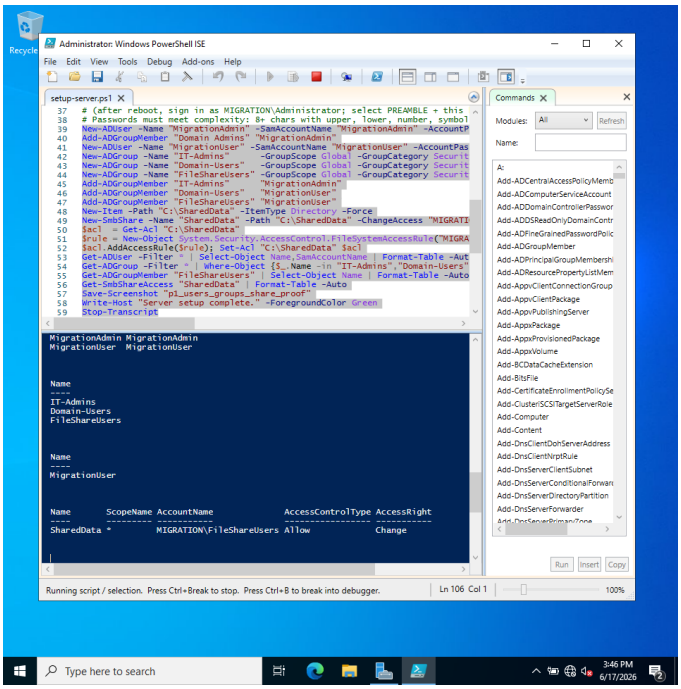

Figure 9: Accounts, security groups, and the SharedData share with permissions

Client Domain Join & Validation

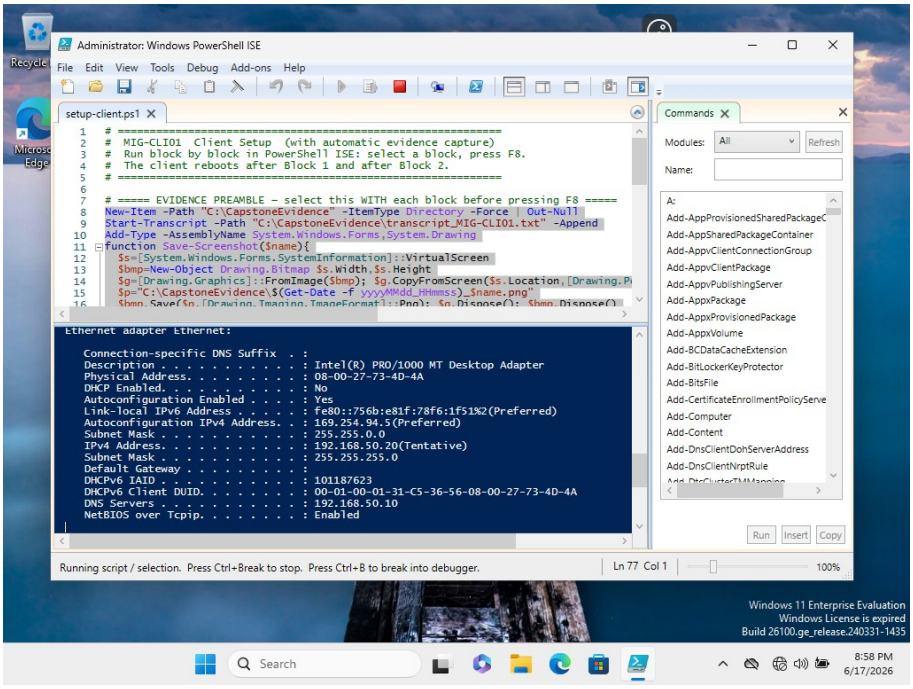

Figure 10: Client static IP 192.168.50.20 and DNS pointing to the domain controller

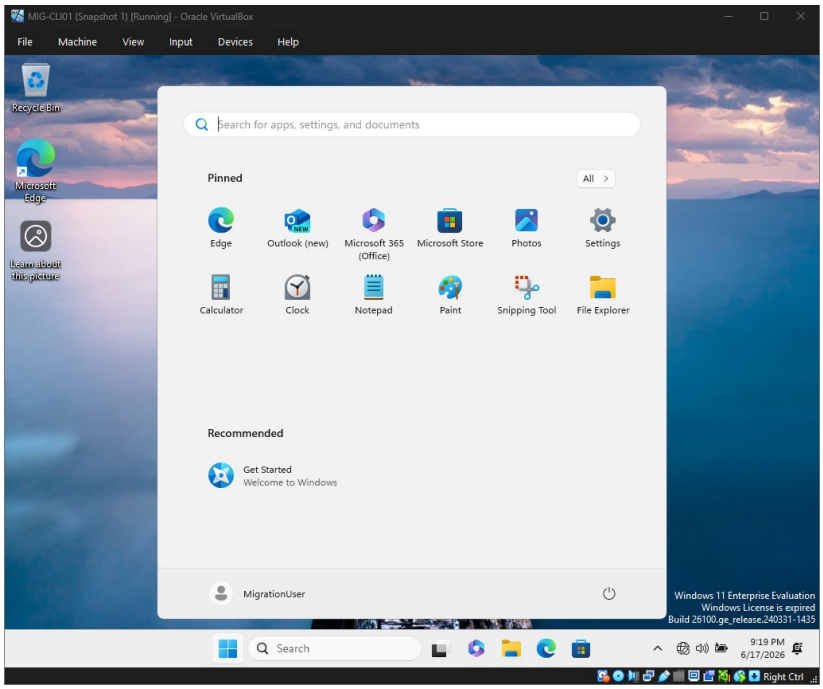

Figure 11: Client signed in as domain account MIGRATION\MigrationUser

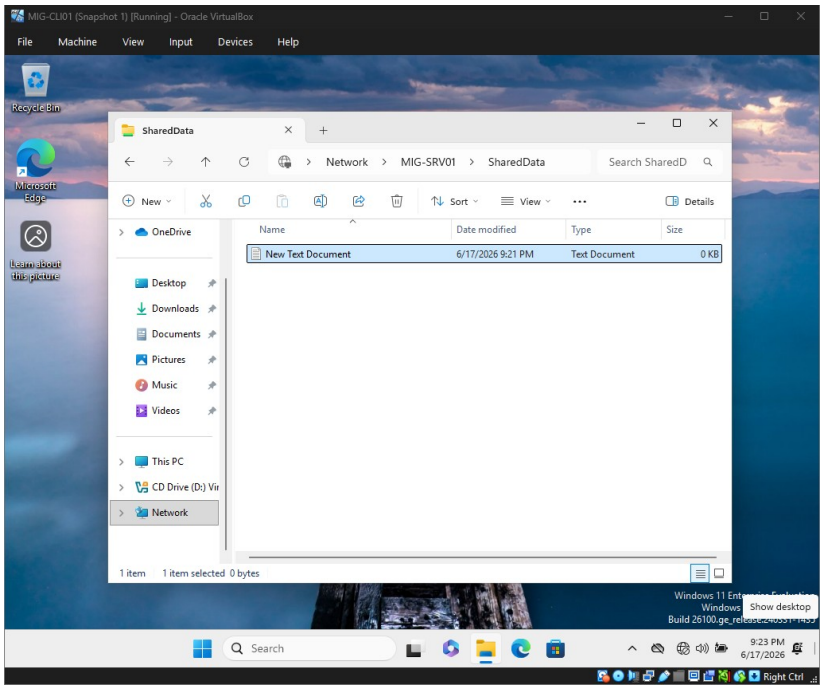

Figure 12: MigrationUser accessing the SharedData network share from the client

Baseline Verification

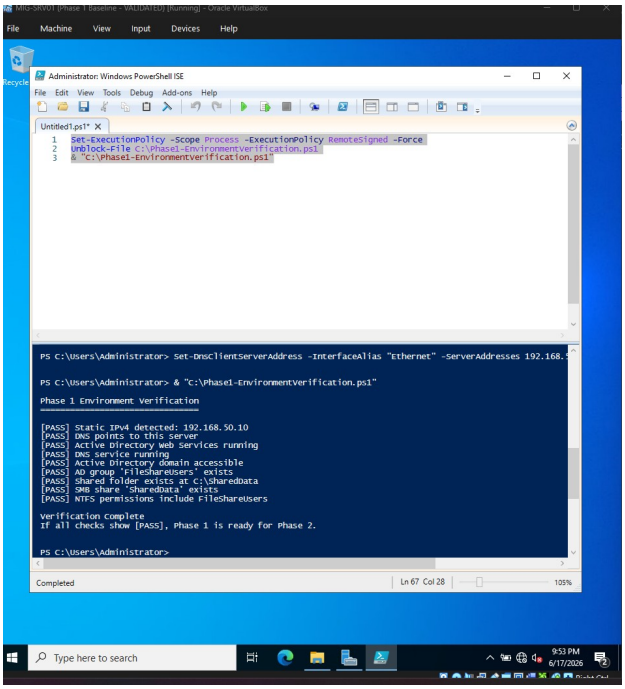

Figure 13: Phase 1 verification script returning PASS on all checks

Phase 2 — Migration

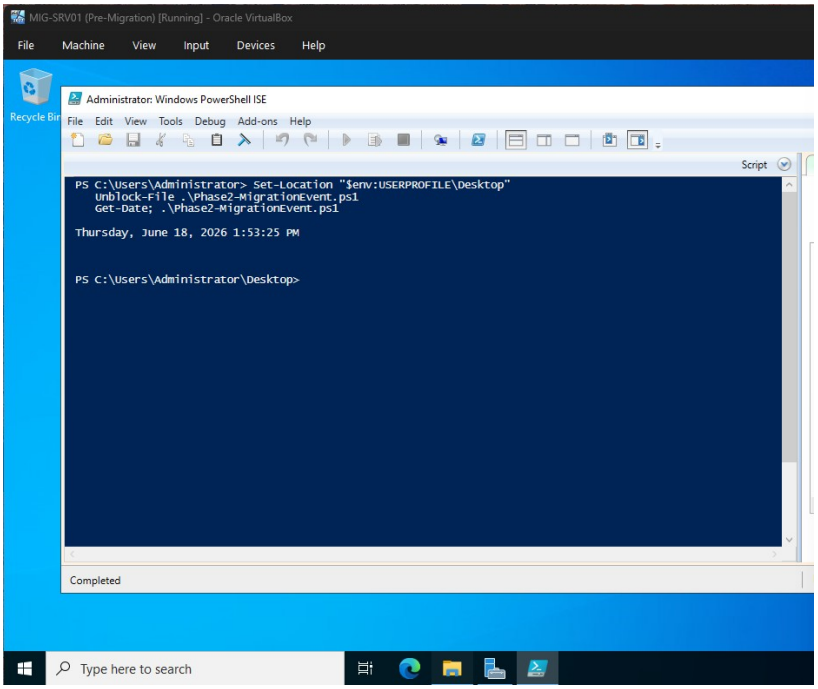

Figure 14: Phase 2 migration script completed on the domain controller

Phase 3 — Incident Response

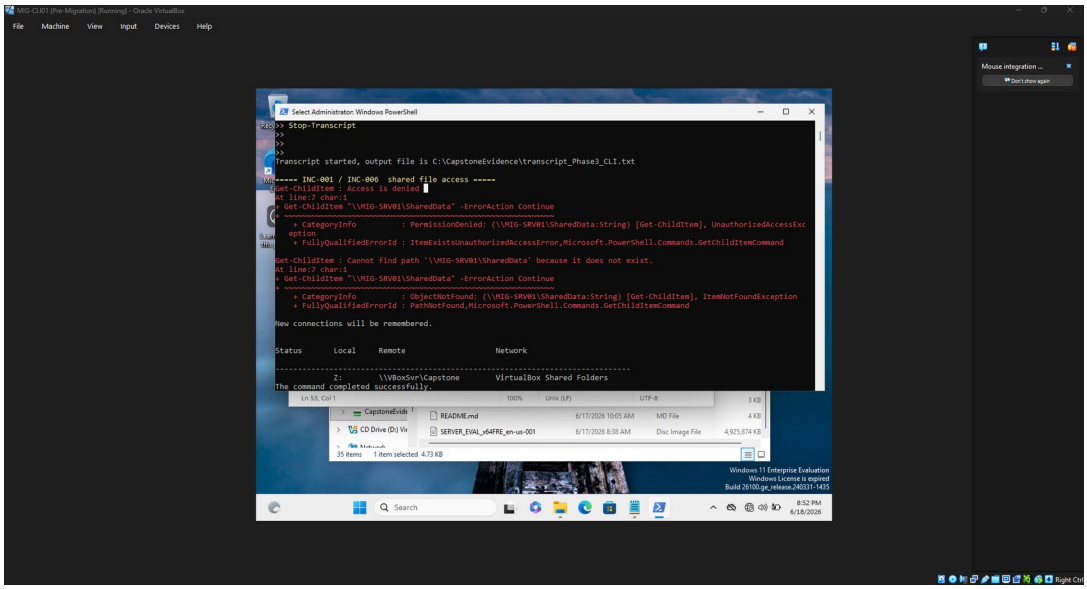

Figure 15: INC-001: SharedData returns Access Denied from the client after migration

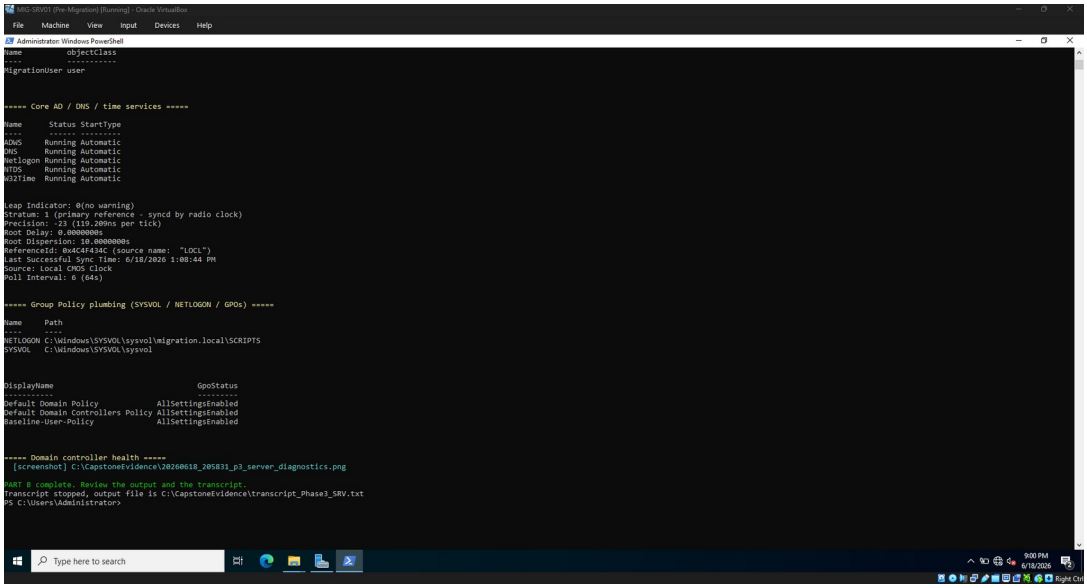

Figure 16: Server diagnostics — services, time, and Group Policy; note the new Baseline-User-Policy GPO

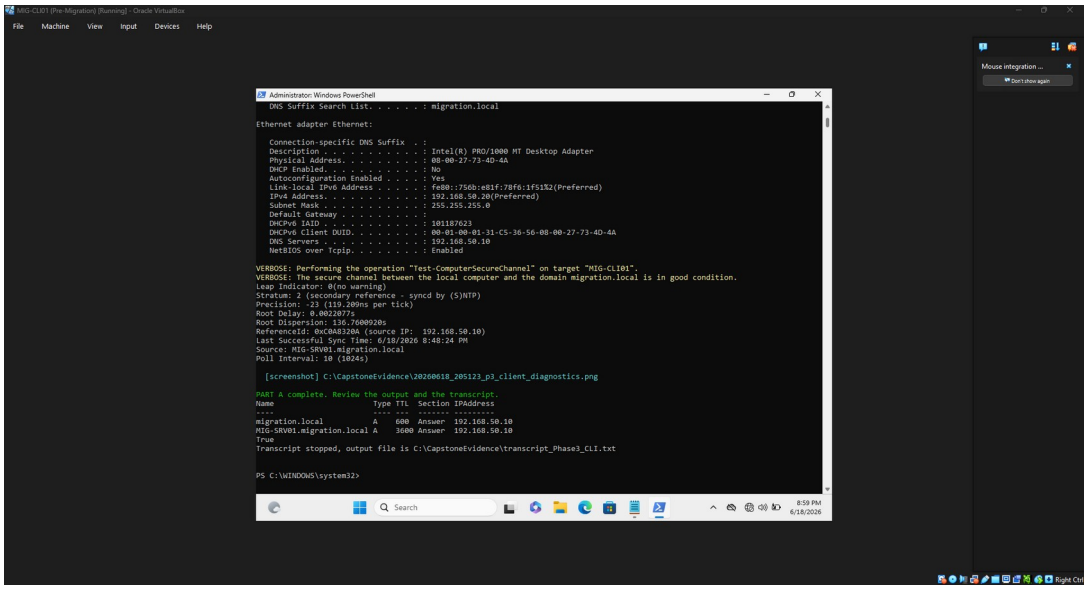

Figure 17: Client diagnostics — DNS, secure channel, and time all healthy

Supplementary — Server Identity

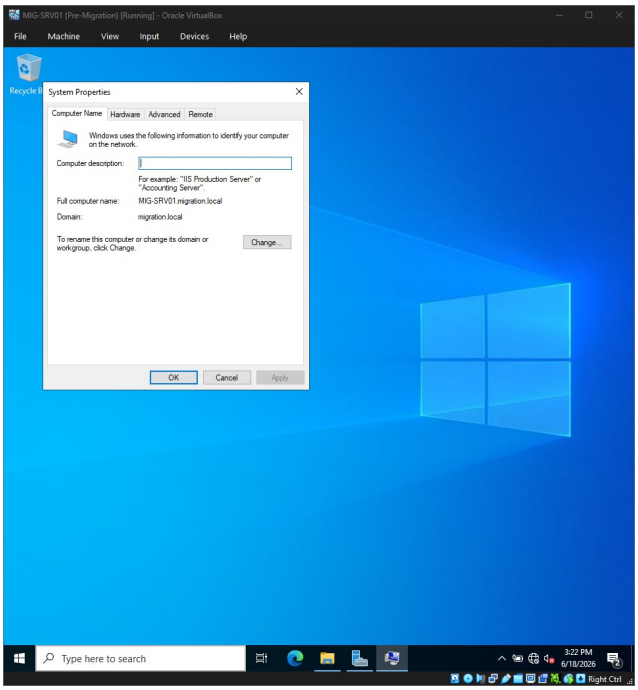

Figure 18: Server System Properties — computer name and domain membership

Appendix B — Network Configuration

Server and client addressing on the isolated internal network (full ipconfig /all output appears in Appendix A, Figures 7 and 10).

Setting	MIG-SRV01 (server)	MIG-CLI01 (client)
IP address	192.168.50.10	192.168.50.20
Subnet mask	255.255.255.0 (/24)	255.255.255.0 (/24)
Preferred DNS	192.168.50.10	192.168.50.10
Adapter	Internal Network	Internal Network
Domain	migration.local	migration.local (joined)

Appendix C — PowerShell Scripts

The reviewed, AI-assisted automation used to build the environment and to run the Phase 3 diagnostics. The setup scripts prompt for passwords at runtime, auto-detect the network adapter, and capture evidence as they run; all are intended to run inside the lab VMs only.

MIG-SRV01 — Server Setup

```
# ################################################################
# ##  RUN INSIDE THE CAPSTONE VM ONLY  —  DO NOT RUN ON YOUR HOST  ##
# ################################################################
#
#  MIG-SRV01  Server Setup  (with automatic evidence capture)
#  1024.3 AI-Enabled IT Support Capstone  -  Phase 1 Environment Build
#  Author: Amanda Kondrat'yev
#
#  WHAT THE EVIDENCE CAPTURE DOES:
#   - Logs every command + output to  C:\CapstoneEvidence\transcript_MIG-SRV01.txt
#   - Saves a full-screen PNG after key steps to  C:\CapstoneEvidence\
#  STILL MANUAL (script can't see the host): VirtualBox VM settings,
#  snapshot list, and the internal-network screen. Capture those yourself.
#
#  HOW TO RUN:
#   - Windows PowerShell as Administrator, INSIDE the VM.
#   - Run ONE BLOCK at a time; the server reboots between blocks.
#   - Paste the evidence preamble at the top of EACH block (a reboot
#     starts a fresh session, so the helper must be re-defined).
#   - Review each command and verify the results before relying on it.
#   - The script auto-detects the active network adapter (no need to hardcode "Ethernet").
# ################################################################



# ===== EVIDENCE PREAMBLE — paste at the start of EVERY block =====
New-Item -Path "C:\CapstoneEvidence" -ItemType Directory -Force | Out-Null
Start-Transcript -Path "C:\CapstoneEvidence\transcript_MIG-SRV01.txt" -Append
Add-Type -AssemblyName System.Windows.Forms,System.Drawing
function Save-Screenshot($name){
  $s=[System.Windows.Forms.SystemInformation]::VirtualScreen
  $bmp=New-Object Drawing.Bitmap $s.Width,$s.Height
  $g=[Drawing.Graphics]::FromImage($bmp); $g.CopyFromScreen($s.Location,[Drawing.Point]::Empty,$s.Size)
  $p="C:\CapstoneEvidence\$(Get-Date -f yyyyMMdd_HHmmss)_$name.png"
  $bmp.Save($p,[Drawing.Imaging.ImageFormat]::Png); $g.Dispose(); $bmp.Dispose()
  Write-Host "  [screenshot saved] $p" -ForegroundColor Cyan
}
# ================================================================



# ---------- BLOCK 1 : static IP + DNS + rename (server reboots) ----------
if ((Read-Host "Type YES only if you are INSIDE the MIG-SRV01 VM") -ne "YES") { Write-Warning "Aborted."; Stop-Transcript; return }
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned -Force


# auto-detect the active network adapter (instead of assuming "Ethernet")
$IfAlias = (Get-NetAdapter -Physical | Where-Object { $_.Status -eq 'Up' } | Select-Object -First 1).Name
if (-not $IfAlias) { $IfAlias = (Get-NetAdapter | Select-Object -First 1).Name }
Write-Host "Using network adapter: $IfAlias" -ForegroundColor Cyan


New-NetIPAddress -InterfaceAlias $IfAlias -IPAddress 192.168.50.10 -PrefixLength 24
Set-DnsClientServerAddress -InterfaceAlias $IfAlias -ServerAddresses 192.168.50.10
ipconfig /all
Save-Screenshot "p1_server_ipconfig"
Stop-Transcript
Rename-Computer -NewName "MIG-SRV01" -Restart



# ---------- BLOCK 2 : install AD DS + promote to DC (server reboots) ----------
# (re-paste the EVIDENCE PREAMBLE first)
Install-WindowsFeature AD-Domain-Services -IncludeManagementTools
Save-Screenshot "p1_adds_feature_installed"
Import-Module ADDSDeployment
Install-ADDSForest -DomainName "migration.local" -DomainNetbiosName "MIGRATION" -InstallDns `
  -SafeModeAdministratorPassword (Read-Host "Enter a DSRM recovery password" -AsSecureString) -Force
# (server reboots automatically after promotion)
```

```
# ---------- BLOCK 3 : users, groups, file share ----------
# (re-paste the EVIDENCE PREAMBLE first; sign in as MIGRATION\Administrator)
# NOTE: passwords must meet AD complexity (8+ chars, upper+lower+number+symbol).
New-ADUser -Name "MigrationAdmin" -SamAccountName "MigrationAdmin" -AccountPassword (Read-Host "Set MigrationAdmin
password" -AsSecureString) -Enabled $true
Add-ADGroupMember "Domain Admins" "MigrationAdmin"
New-ADUser -Name "MigrationUser" -SamAccountName "MigrationUser" -AccountPassword (Read-Host "Set MigrationUser password"
-AsSecureString) -Enabled $true
New-ADGroup -Name "IT-Admins"      -GroupScope Global -GroupCategory Security
New-ADGroup -Name "Domain-Users"   -GroupScope Global -GroupCategory Security
New-ADGroup -Name "FileShareUsers" -GroupScope Global -GroupCategory Security
Add-ADGroupMember "IT-Admins"      "MigrationAdmin"
Add-ADGroupMember "Domain-Users"   "MigrationUser"
Add-ADGroupMember "FileShareUsers" "MigrationUser"


New-Item -Path "C:\SharedData" -ItemType Directory -Force
New-SmbShare -Name "SharedData" -Path "C:\SharedData" -ChangeAccess "MIGRATION\FileShareUsers"
$acl  = Get-Acl "C:\SharedData"
$rule = New-Object
System.Security.AccessControl.FileSystemAccessRule("MIGRATION\FileShareUsers","Modify","ContainerInherit,ObjectInherit","None",
"Allow")
$acl.AddAccessRule($rule); Set-Acl "C:\SharedData" $acl


# print the proof into the transcript, then screenshot it
Get-ADUser -Filter * | Select-Object Name,SamAccountName | Format-Table -Auto
Get-ADGroup -Filter * | Where-Object {$_.Name -in "IT-Admins","Domain-Users","FileShareUsers"} | Select-Object Name | Format-Table
-Auto
Get-ADGroupMember "FileShareUsers" | Select-Object Name | Format-Table -Auto
Get-SmbShareAccess "SharedData" | Format-Table -Auto
Save-Screenshot "p1_users_groups_share_proof"
Write-Host "Server setup complete." -ForegroundColor Green
Stop-Transcript


# ################################################################
#  Script author: Amanda Kondrat'yev  -  1024.3 AI-Enabled IT Support Capstone
# ################################################################
```

MIG-CLI01 — Client Setup

```
# ################################################################
# ##  RUN INSIDE THE CAPSTONE VM ONLY  —  DO NOT RUN ON YOUR HOST  ##
# ################################################################
#
#  MIG-CLI01  Client Setup  (with automatic evidence capture)
#  1024.3 AI-Enabled IT Support Capstone  -  Phase 1 Environment Build
#  Author: Amanda Kondrat'yev
#
#  Evidence: logs to C:\CapstoneEvidence\transcript_MIG-CLI01.txt and saves PNGs.
#  Manual (host-side): VirtualBox settings + snapshots.
#  Requires Windows 11 Pro/Enterprise (Home cannot join a domain).
#  Run as Administrator, one block at a time; re-paste the preamble each block.
#  The script auto-detects the active network adapter (no need to hardcode "Ethernet").
# ################################################################



# ===== EVIDENCE PREAMBLE — paste at the start of EVERY block =====
New-Item -Path "C:\CapstoneEvidence" -ItemType Directory -Force | Out-Null
Start-Transcript -Path "C:\CapstoneEvidence\transcript_MIG-CLI01.txt" -Append
Add-Type -AssemblyName System.Windows.Forms,System.Drawing
function Save-Screenshot($name){
  $s=[System.Windows.Forms.SystemInformation]::VirtualScreen
  $bmp=New-Object Drawing.Bitmap $s.Width,$s.Height
  $g=[Drawing.Graphics]::FromImage($bmp); $g.CopyFromScreen($s.Location,[Drawing.Point]::Empty,$s.Size)
  $p="C:\CapstoneEvidence\$(Get-Date -f yyyyMMdd_HHmmss)_$name.png"
  $bmp.Save($p,[Drawing.Imaging.ImageFormat]::Png); $g.Dispose(); $bmp.Dispose()
  Write-Host "  [screenshot saved] $p" -ForegroundColor Cyan
}
# ================================================================



# ---------- BLOCK 1 : static IP + DNS + rename (client reboots) ----------
if ((Read-Host "Type YES only if you are INSIDE the MIG-CLI01 VM") -ne "YES") { Write-Warning "Aborted."; Stop-Transcript; return }
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned -Force
```

```
# auto-detect the active network adapter (instead of assuming "Ethernet")
$IfAlias = (Get-NetAdapter -Physical | Where-Object { $_.Status -eq 'Up' } | Select-Object -First 1).Name
if (-not $IfAlias) { $IfAlias = (Get-NetAdapter | Select-Object -First 1).Name }
Write-Host "Using network adapter: $IfAlias" -ForegroundColor Cyan


New-NetIPAddress -InterfaceAlias $IfAlias -IPAddress 192.168.50.20 -PrefixLength 24
Set-DnsClientServerAddress -InterfaceAlias $IfAlias -ServerAddresses 192.168.50.10
ipconfig /all
Save-Screenshot "p1_client_ipconfig"
Stop-Transcript
Rename-Computer -NewName "MIG-CLI01" -Restart



# ---------- BLOCK 2 : join the domain (client reboots) ----------
# (re-paste the EVIDENCE PREAMBLE first; sign in as MIGRATION\MigrationAdmin when prompted)
Add-Computer -DomainName "migration.local" -Credential (Get-Credential "MIGRATION\MigrationAdmin") -Restart



# ---------- BLOCK 3 : validate (sign in as MIGRATION\MigrationUser) ----------
# (re-paste the EVIDENCE PREAMBLE first)
ping MIG-SRV01
nslookup migration.local
Test-Path "\\MIG-SRV01\SharedData"
Save-Screenshot "p1_client_validation"
Start-Process "\\MIG-SRV01\SharedData"
Start-Sleep -Seconds 3
Save-Screenshot "p1_client_share_open"
Stop-Transcript


# ################################################################
#  Script author: Amanda Kondrat'yev  -  1024.3 AI-Enabled IT Support Capstone
# ################################################################
```

Phase3-Investigation.ps1 — Read-only Diagnostics

```
# ################################################################
# ##  RUN INSIDE THE CAPSTONE VMs  —  READ-ONLY DIAGNOSTICS  ##
# ################################################################
#
#  Phase3-Investigation.ps1   (with automatic evidence capture)
#  1024.3 AI-Enabled IT Support Capstone  -  Amanda Kondrat'yev
#
#  Phase 3 is INVESTIGATE-ONLY. These commands gather evidence and
#  CHANGE NOTHING. Run PART A on the client, PART B on the server.
#  Everything is logged to C:\CapstoneEvidence and screenshotted.
#  Read the output yourself and decide the root cause - the script
#  does not diagnose for you.
# ################################################################



# ========================== PART A : RUN ON MIG-CLI01 (client) ==========================
# --- evidence preamble ---
New-Item -Path "C:\CapstoneEvidence" -ItemType Directory -Force | Out-Null
Start-Transcript -Path "C:\CapstoneEvidence\transcript_Phase3_CLI.txt" -Append
Add-Type -AssemblyName System.Windows.Forms,System.Drawing
function Save-Screenshot($name){ $s=[System.Windows.Forms.SystemInformation]::VirtualScreen; $bmp=New-Object Drawing.Bitmap
$s.Width,$s.Height; $g=[Drawing.Graphics]::FromImage($bmp); $g.CopyFromScreen($s.Location,[Drawing.Point]::Empty,$s.Size);
$p="C:\CapstoneEvidence\$(Get-Date -f yyyyMMdd_HHmmss)_$name.png"; $bmp.Save($p,[Drawing.Imaging.ImageFormat]::Png);
$g.Dispose(); $bmp.Dispose(); Write-Host "  [screenshot] $p" -ForegroundColor Cyan }


Write-Host "`n===== INC-001 / INC-006  shared file access =====" -ForegroundColor Yellow
Get-ChildItem "\\MIG-SRV01\SharedData" -ErrorAction Continue
net use


Write-Host "`n===== INC-002 / INC-003  logon speed & Group Policy =====" -ForegroundColor Yellow
gpresult /r
gpresult /h "C:\CapstoneEvidence\gpresult_client.html" /f
Get-WinEvent -LogName "Microsoft-Windows-GroupPolicy/Operational" -MaxEvents 25 -ErrorAction SilentlyContinue |
  Select-Object TimeCreated,Id,LevelDisplayName,Message | Format-Table -Wrap


Write-Host "`n===== INC-004  printer =====" -ForegroundColor Yellow
```

```
Get-Printer -ErrorAction SilentlyContinue | Format-Table Name,DriverName,PortName -Auto
Get-Service Spooler | Format-Table Name,Status,StartType -Auto


Write-Host "`n===== INC-005  performance / services =====" -ForegroundColor Yellow
Get-Service | Where-Object {$_.Status -ne 'Running' -and $_.StartType -eq 'Automatic'} | Format-Table Name,Status,StartType -Auto
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 Name,CPU,WS | Format-Table -Auto


Write-Host "`n===== INC-006  DNS / secure channel / time =====" -ForegroundColor Yellow
ipconfig /all
Resolve-DnsName "migration.local" -ErrorAction SilentlyContinue
Resolve-DnsName "MIG-SRV01" -ErrorAction SilentlyContinue
Test-ComputerSecureChannel -Verbose
w32tm /query /status


Save-Screenshot "p3_client_diagnostics"
Write-Host "`nPART A complete. Review the output and the transcript." -ForegroundColor Green
Stop-Transcript



# ========================= PART B : RUN ON MIG-SRV01 (server) =========================
# (re-paste this preamble on the server)
New-Item -Path "C:\CapstoneEvidence" -ItemType Directory -Force | Out-Null
Start-Transcript -Path "C:\CapstoneEvidence\transcript_Phase3_SRV.txt" -Append
Add-Type -AssemblyName System.Windows.Forms,System.Drawing
function Save-Screenshot($name){ $s=[System.Windows.Forms.SystemInformation]::VirtualScreen; $bmp=New-Object Drawing.Bitmap
$s.Width,$s.Height; $g=[Drawing.Graphics]::FromImage($bmp); $g.CopyFromScreen($s.Location,[Drawing.Point]::Empty,$s.Size);
$p="C:\CapstoneEvidence\$(Get-Date -f yyyyMMdd_HHmmss)_$name.png"; $bmp.Save($p,[Drawing.Imaging.ImageFormat]::Png);
$g.Dispose(); $bmp.Dispose(); Write-Host "  [screenshot] $p" -ForegroundColor Cyan }


Write-Host "`n===== INC-001 / INC-006  share & NTFS permissions =====" -ForegroundColor Yellow
Get-SmbShareAccess "SharedData" | Format-Table -Auto
(Get-Acl "C:\SharedData").Access | Format-Table IdentityReference,FileSystemRights,AccessControlType -Auto
Get-ADGroupMember "FileShareUsers" -ErrorAction SilentlyContinue | Format-Table Name,objectClass -Auto


Write-Host "`n===== Core AD / DNS / time services =====" -ForegroundColor Yellow
Get-Service ADWS,DNS,Netlogon,NTDS,W32Time -ErrorAction SilentlyContinue | Format-Table Name,Status,StartType -Auto
w32tm /query /status


Write-Host "`n===== Group Policy plumbing (SYSVOL / NETLOGON / GPOs) =====" -ForegroundColor Yellow
Get-SmbShare | Where-Object {$_.Name -in "SYSVOL","NETLOGON"} | Format-Table Name,Path -Auto
Get-GPO -All -ErrorAction SilentlyContinue | Select-Object DisplayName,GpoStatus | Format-Table -Auto


Write-Host "`n===== Domain controller health =====" -ForegroundColor Yellow
dcdiag /q


Save-Screenshot "p3_server_diagnostics"
Write-Host "`nPART B complete. Review the output and the transcript." -ForegroundColor Green
Stop-Transcript
```

Phase4-Verification.ps1 — Fix Verification

```
# ##############################################################
# ##  RUN INSIDE THE CAPSTONE VMs  —  FIX VERIFICATION  ##
# ##############################################################
#
#  Phase4-Verification.ps1   (with automatic evidence capture)
#  1024.3 AI-Enabled IT Support Capstone  -  Amanda Kondrat'yev
#
#  YOUR FIXES come from your Phase 3 evidence - decide and apply them
#  yourself (no blind/shotgun fixes). This script RE-TESTS each ticket
#  so you can capture proof. Run a ticket's block BEFORE your fix and
#  AGAIN after, to show the change. All output -> C:\CapstoneEvidence.
#  Most blocks run on the CLIENT; share/permission checks on the SERVER.
# ##############################################################


# --- evidence preamble (re-paste each session / machine) ---
New-Item -Path "C:\CapstoneEvidence" -ItemType Directory -Force | Out-Null
Start-Transcript -Path "C:\CapstoneEvidence\transcript_Phase4.txt" -Append
Add-Type -AssemblyName System.Windows.Forms,System.Drawing
function Save-Screenshot($name){ $s=[System.Windows.Forms.SystemInformation]::VirtualScreen; $bmp=New-Object Drawing.Bitmap
$s.Width,$s.Height; $g=[Drawing.Graphics]::FromImage($bmp); $g.CopyFromScreen($s.Location,[Drawing.Point]::Empty,$s.Size);
$p="C:\CapstoneEvidence\$(Get-Date -f yyyyMMdd_HHmmss)_$name.png"; $bmp.Save($p,[Drawing.Imaging.ImageFormat]::Png);
$g.Dispose(); $bmp.Dispose(); Write-Host "  [screenshot] $p" -ForegroundColor Cyan }
```

```
# ===== INC-001 / INC-006  file access (client; perms on server) =====
Write-Host "`n[Verify INC-001/006] shared file access" -ForegroundColor Yellow
Get-ChildItem "\\MIG-SRV01\SharedData"            # CLIENT: should now list contents
# On SERVER instead: Get-SmbShareAccess "SharedData"; (Get-Acl "C:\SharedData").Access | ft -Auto
Save-Screenshot "p4_inc001_006_fileaccess"


# ===== INC-002 / INC-003  logon & Group Policy (client) =====
Write-Host "`n[Verify INC-002/003] Group Policy applies" -ForegroundColor Yellow
gpupdate /force
gpresult /r
Save-Screenshot "p4_inc002_003_grouppolicy"


# ===== INC-004  printer (client) =====
Write-Host "`n[Verify INC-004] printer present & spooler running" -ForegroundColor Yellow
Get-Service Spooler | Format-Table Name,Status,StartType -Auto
Get-Printer -ErrorAction SilentlyContinue | Format-Table Name,DriverName,PortName -Auto
Save-Screenshot "p4_inc004_printer"


# ===== INC-005  performance / services (client) =====
Write-Host "`n[Verify INC-005] auto services running" -ForegroundColor Yellow
Get-Service | Where-Object {$_.Status -ne 'Running' -and $_.StartType -eq 'Automatic'} | Format-Table Name,Status -Auto
Save-Screenshot "p4_inc005_services"


# ===== INC-006  DNS / secure channel / time (client) =====
Write-Host "`n[Verify INC-006] name resolution & secure channel" -ForegroundColor Yellow
Resolve-DnsName "migration.local" -ErrorAction SilentlyContinue
Test-ComputerSecureChannel -Verbose
w32tm /query /status
Save-Screenshot "p4_inc006_dns_securechannel"


Write-Host "`nVerification pass complete." -ForegroundColor Green
Stop-Transcript
```

Evidence-Helpers.ps1 — Evidence Capture Utility

```
# ============================================================
#  Evidence-Helpers.ps1   (run INSIDE the VM)
#  1024.3 AI-Enabled IT Support Capstone  -  Amanda Kondrat'yev
#
#  Dot-source this at the start of any session to turn on capture:
#      . .\Evidence-Helpers.ps1
#      Start-Evidence "MIG-CLI01"      # names the transcript
#      Save-Screenshot "my_step"       # saves a PNG anytime
#  Everything lands in  C:\CapstoneEvidence\
# ============================================================
New-Item -Path "C:\CapstoneEvidence" -ItemType Directory -Force | Out-Null
Add-Type -AssemblyName System.Windows.Forms,System.Drawing


function Start-Evidence([string]$tag="session"){
  Start-Transcript -Path "C:\CapstoneEvidence\transcript_$tag.txt" -Append
}
function Save-Screenshot([string]$name){
  $s=[System.Windows.Forms.SystemInformation]::VirtualScreen
  $bmp=New-Object Drawing.Bitmap $s.Width,$s.Height
  $g=[Drawing.Graphics]::FromImage($bmp); $g.CopyFromScreen($s.Location,[Drawing.Point]::Empty,$s.Size)
  $p="C:\CapstoneEvidence\$(Get-Date -f yyyyMMdd_HHmmss)_$name.png"
  $bmp.Save($p,[Drawing.Imaging.ImageFormat]::Png); $g.Dispose(); $bmp.Dispose()
  Write-Host "  [screenshot saved] $p" -ForegroundColor Cyan
}
Write-Host "Evidence helpers loaded. Use Start-Evidence and Save-Screenshot." -ForegroundColor Green
```